

ESTsoft ALZ 파일 검증기

확장자-매직넘버 이중 검증을 우회하는 파일 업로드 취약점 공격 (Extension & Magic Number Bypass to Arbitrary File Upload)

TARGET	problem07	SEVERITY	MEDIUM
PREPARED FOR	8low8lowme Team	STATUS	FINAL
AUTHOR	김혜미	CWE	CWE-434 · CWE-646

1. 개요 (Executive Summary)

problem07(ESTsoft ALZ 파일 검증기)를 분석한 결과, 서버가 업로드 파일에 대해 확장자 검증과 매직넘버 (파일 시그니처) 검증이라는 두 단계 방어를 적용하고 있음을 확인했다. 그러나 두 검증 모두 파일의 실제 형식이 아니라 "이름이 .alz로 끝나는가"와 "첫 4바이트가 특정 값과 일치하는가"만을 확인하는 형태였다. 이 두 조건은 파일의 실제 내용/형식과 무관하게 조작이 가능했다. 공격자는 임의의 내용 앞에 정해진 매직넘버 4바이트만 붙이는 것만으로 검증 로직 전체를 우회할 수 있었고, 최종적으로 서버가 반환하는 FLAG를 획득했다. 이 과정에서 서버는 응답 헤더(X-Error-Message)를 통해 어느 검증 단계에서 실패했는지를 그대로 노출하고 있어, 공격자가 검증 로직의 구조를 순차적으로 유추하는 데 활용되었다.

2. 공격 시나리오 (Attack Narrative)

단계	기법	확보한 정보 / 결과
1	기본 파일(.txt) 업로드 시도	"Only .alz files allowed." + 헤더 X-Error-Message: Extension failed 확인
2	.alz로 확장자만 변경 후 업로드	"Invalid file signature." + 헤더 X-Error-Message: ESTsoft signature failed 확인
3	Burp Suite로 응답 헤더 분석	두 단계 검증(확장자 → 시그니처) 구조 파악
4	printf로 매직넘버(41 4C 5A 01) 헤더를 가진 .alz 파일 제작	올바른 시그니처를 가진 위조 파일 생성

5	조작된 파일 업로드	Success! Flag 획득
---	------------	------------------

3. 상세 재현 절차 (Proof of Concept)

3.1 정찰 — 일반 파일 업로드 시도

problem07 페이지(index.php)에는 단순한 파일 업로드 폼이 하나 존재한다. 별도의 안내 문구 없이 파일 선택 후 업로드 버튼만 노출되어 있어, 어떤 파일 형식이 허용되는지는 직접 시도하며 확인해야 한다.



임의의 .txt 파일을 업로드하면 실패 메시지가 반환된다.

3.2 Burp Suite로 응답 헤더 분석

브라우저 화면에는 단순 실패 메시지만 보이지만, Burp Suite로 응답을 가로채면 다음과 같은 커스텀 헤더가 함께 반환되는 것을 확인할 수 있다.



이 헤더를 통해 서버가 1차로 파일 확장자를 검사하고 있으며, 현재 시도가 그 단계에서 걸렸다는 사실을 알 수 있다.

3.3 확장자 우회 — .alz로 이름 변경 후 재시도

파일명만 .alz로 변경하여 다시 업로드한다.

```
test.txt → test.alz
```

이번에는 확장자 검증은 통과했지만, 같은 실패 메시지가 반환된다.

```
X-Error-Message: ESTsoft signature failed
```

두 가지 실패 메시지("Extension failed" → "ESTsoft signature failed")의 차이를 통해, 서버가 확장자 검증 이후 파일 내용(시그니처)까지 별도로 검사하는 2단계 검증 구조라는 사실을 확정할 수 있다.

3.4 매직넘버(시그니처) 조작 파일 제작

ALZ 파일의 공식 매직넘버는 첫 4바이트가 41 4C 5A 01(16진수)이다. 이 값을 파일 맨 앞에 삽입하면 시그니처 검증을 통과할 수 있다. printf 명령어로 hexa 에디터 없이 바로 생성한다.

```
$ printf "\x41\x4C\x5A\x01" > test.alz
```

```
# 생성된 파일 확인
```

```
$ xxd test.alz
```

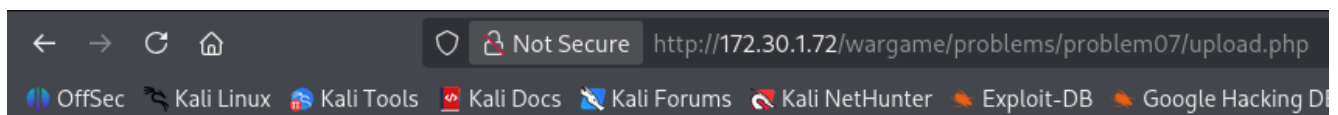
```
00000000: 414c 5a01
```

```
ALZ.
```

실제 파일 내용은 ALZ 압축 파일과 전혀 무관하지만, 서버는 앞 4바이트만 검사하기 때문에 이 조작된 파일도 정상 ALZ 파일로 판단하게 된다.

3.5 업로드 성공 및 FLAG 획득

조작된 test.alz 파일을 업로드 폼에 제출하면 두 검증(확장자, 시그니처)을 모두 통과하여 서버가 FLAG를 응답 본문에 그대로 반환한다.



Success! Flag is: **FLAG{MGQyNTElK2EjdGU4bA}**

4. 근본 원인 분석 (Root Cause)

- 확장자 검증(pathinfo)과 시그니처 검증(첫 4바이트 비교) 모두 파일의 이름/앞부분 값만 확인할 뿐, 파일 전체의 실제 형식·무결성을 검증하지 않음
- 매직넘버 검사는 "확장자만 검사하는 것보다 안전하다"는 통념과 달리, 검사 대상이 파일 앞부분 몇 바이트로 한정되면 나머지 내용은 임의로 채울 수 있는 Polyglot 공격에 그대로 노출됨
- 검증 실패 사유를 X-Error-Message 응답 헤더에 그대로 노출하여, 공격자가 서버의 검증 순서와 로직 구조를 손쉽게 역추적할 수 있는 사이드 채널을 제공함
- 업로드된 파일이 실행 가능한 위치에 저장되거나 별도의 콘텐츠 재검증 없이 서비스된다면, 이 우회 기법은 단순 정책 우회를 넘어 악성 파일 유포·원격 코드 실행으로 이어질 수 있는 체인의 시작점이 될 수 있음